Week 1 - Domain 1: SDLC Automation - CI/CD Concepts, Testing, Artifacts and Third-Party CI

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

These are the Part 1 items from the official exam guide that the service topics didn't fully cover.

Section	Exam guide task	What it covers
8.1	Task 1.1	CI vs continuous delivery vs continuous deployment, SDLC phases, branching, feature flags, environment promotion, single vs multi-account
8.2	Task 1.2	Test types per stage, exit codes, load testing, Device Farm, Synthetics canaries, Step Functions / Lambda tests, chaos tests
8.3	Task 1.3	S3 as an artifact store, Object Lock, build once deploy many, artifact retention, code signing
8.4	Task 1.1 / 1.3	GitHub Actions OIDC, Jenkins, IAM Roles Anywhere, custom actions
8.5 - 8.7		Exam traps, hands-on lab, check yourself

Part 8: CI/CD Concepts, Testing, Artifacts and Third-Party CI

8.1 CI/CD and SDLC concepts

SDLC phases: plan > code > build > test > release > deploy > operate > monitor (and feed back into plan).

Term	Meaning	Exam signal
Continuous integration (CI)	Developers merge small changes often; every change is **built and tested automatically**	"Find integration problems early", "build and unit test on every commit / PR"
Continuous delivery	Every change that passes the pipeline is **ready to release**; production deploy needs a **manual approval**	"Manual approval before production"
Continuous deployment	Every change that passes all tests goes to **production automatically**, no human step	"No manual intervention", "deploy to prod automatically"

Branching strategies:

Strategy	How it works	Fits
Trunk-based development	Everyone commits to main (or very short-lived branches, merged within a day); unfinished work hidden behind **feature flags**	CI / continuous deployment, fast teams
GitHub flow	Short feature branch > pull request > review + checks > merge to main > deploy	Most modern teams
GitFlow	Long-lived develop, release and hotfix branches; release branches stabilise	Scheduled releases, slower cadence

- **Feature flags decouple deploying from releasing:** code is deployed but switched off; turn it on for some users later (dark launch). On AWS: **AWS AppConfig feature flags** (with validators and gradual rollout strategies and automatic rollback on CloudWatch alarms).
- **Pull request checks** (build, unit tests, static analysis, CodeGuru Reviewer) before merge protect main.

Environment promotion:

- The same pipeline moves a change through **Dev > Test / Staging > Prod**; each stage adds tests and gates (approvals, alarms, deployment windows).
- **Build once, deploy many** (see 8.3): the exact artifact tested in staging is the one deployed to prod.

Single-account vs multi-account pipelines:

	Single account	Multi-account (recommended for production)
Setup	Pipeline and all environments in one account	Separate Dev / Test / Prod accounts (AWS Organizations, Control Tower) + a central Tools / DevOps account running pipelines
Isolation	A mistake in dev can affect prod; shared quotas	Strong security and blast-radius boundaries; separate quotas and billing
Needs	Simple IAM	Cross-account roles, customer-managed KMS key, bucket policies (see CodePipeline)
Exam signal	Small team, simple	"Developer mistake deleted prod resources", "isolate environments", "least privilege per environment"

8.2 Testing in the pipeline

Test types and where they belong:

Test type	What it checks	Pipeline stage	AWS tools
Unit tests	Single functions / classes, no external dependencies	Build (every commit and PR)	CodeBuild + report groups
Static analysis / lint / SAST	Code quality, security issues, secrets in code	Build / PR	CodeGuru Reviewer, Amazon Q Developer, linters in CodeBuild
Dependency / image scanning	Known CVEs in packages and images	Build / after push	Amazon Inspector, ECR scanning
Integration tests	Components working together (DB, queues, APIs)	After deploying to a test environment	CodeBuild in a VPC
Contract / API tests	API stays compatible with consumers	Test stage	CodeBuild
Acceptance / end-to-end / UI tests	Whole user journeys, browsers, mobile devices	Staging	CodeBuild (Selenium / Playwright), AWS Device Farm
Performance / load / stress tests	Behaviour under heavy load, breaking point, benchmarks	Staging (production-like)	Distributed Load Testing on AWS, third-party (BlazeMeter)
Resilience / chaos tests	Behaviour when parts fail	Staging (carefully in prod)	AWS Fault Injection Service (FIS)
Smoke tests	"Is the new version up and basically working?"	Right after each deploy	CodeBuild, CodeDeploy ValidateService hook
Synthetic monitoring	User journeys keep working in production	Production, continuously	CloudWatch Synthetics canaries

- **Fast tests first** (unit, static) so developers get quick feedback; slow, expensive tests later on production-like environments.
- **Test pyramid:** many unit tests, fewer integration tests, few end-to-end tests.

Exit codes decide pass / fail (exam guide: "measure application health based on exit codes"):

- In CodeBuild, every command's **exit code** counts: `0` = success, **non-zero = the command fails and the phase fails**.
- Test runners (pytest, JUnit / Maven, npm test) return non-zero when a test fails, which fails the build.
- **Traps that hide failures:**
  - `pytest || true` - always exits 0, so failed tests pass the build.
  - Pipes: `pytest | tee out.log` - the exit code is from `tee`. Use `set -o pipefail`.
  - `curl https://app/health` - exits 0 even for HTTP 500. Use `curl --fail` (non-zero on HTTP errors).
- **CodeDeploy hook scripts** (e.g. `ValidateService`): a non-zero exit code **fails the deployment** (and can trigger rollback).
- **Lambda test actions** report with `PutJobSuccessResult` / `PutJobFailureResult`.

- **Container health checks** (ECS `healthCheck` command): exit 0 = healthy, non-zero = unhealthy, so the task is replaced.

Load and stress testing at scale:

- **Distributed Load Testing on AWS** (an AWS Solution): runs load tests (JMeter / Taurus / k6 scripts) from containers on **Fargate**, optionally from **several Regions**, with a web console and API so a pipeline stage can start a test and check results.
- Third-party test actions in CodePipeline (e.g. BlazeMeter).
- Run against **staging**, not production; check service quotas and tell AWS about very large tests if required by policy.

AWS Device Farm:

- Tests apps on **real mobile devices** (Android, iOS) and **desktop browsers** (Selenium grid).
- CodePipeline has a **Device Farm test action**.

CloudWatch Synthetics canaries:

- Scripts (Node.js / Python, Puppeteer / Selenium) that run **on a schedule** and act like a user: load pages, call APIs, check results, take screenshots.
- Use **after deployment** and continuously in production. A **CloudWatch alarm on canary failures** can trigger **CodeDeploy automatic rollback** or a **CodePipeline V2 rollback condition**.

Calling AWS services from the pipeline for testing:

- **Lambda invoke action** - custom checks (call an API, compare results); must call `PutJobSuccessResult` / `PutJobFailureResult`.
- **Step Functions invoke action** - orchestrate **complex test workflows** (parallel tests, retries, waits, branching, human steps).
- **CodeBuild test action** - any test runner, with report groups.
- **AWS FIS** experiments started from CodeBuild / Lambda / Step Functions for chaos testing in staging.

8.3 Artifact management

S3 as an artifact repository:

- **Versioning** - keeps every version of an artifact (required for a CodePipeline S3 source).
- **Lifecycle rules** - move old artifacts to cheaper storage classes and **expire noncurrent versions** after N days to control cost.
- **Encryption** - SSE-KMS (customer-managed key for cross-account), bucket policy that **denies non-HTTPS** and unencrypted uploads, **Block Public Access**.
- **S3 Object Lock** - **WORM** (write once, read many): objects can't be deleted or overwritten for a retention period. Needs versioning.
  - **Governance mode:** users with a special permission can override.
  - **Compliance mode: nobody** (not even root) can delete before the retention date - for strict audit rules.
  - **Legal hold:** no expiry date until removed.
- **Replication** (CRR / SRR) for DR copies of release artifacts.
- **Presigned URLs** to give temporary download access to an artifact.

Build once, deploy many:

- Build the artifact **once** (JAR, ZIP, container image), then **promote the same artifact** through Test > Staging > Prod.
- **Never rebuild for production** - a rebuild can pull different dependency versions, so prod would run untested code.
- Keep **environment configuration outside the artifact**: Parameter Store, Secrets Manager, AppConfig, CloudFormation TemplateConfiguration files.
- Identify artifacts immutably: **semantic version or commit ID**, ECR **image digest**, S3 version ID, CodeArtifact package version (`copy-package-versions` to promote).

Artifact retention across services:

Store	How to control retention
CodePipeline artifact bucket	S3 lifecycle rule (pipeline never deletes artifacts)
S3 release artifacts	Lifecycle (transition / expire noncurrent), Object Lock for must-keep
ECR images	Lifecycle policies (untagged, count, age)
CodeArtifact packages	Version status (Unlisted, Archived), delete or dispose versions
CodeBuild reports	30 days; export raw files to S3 to keep longer

Creating artifacts: mostly **CodeBuild** (compile, package, docker build); **Lambda** can also package or transform artifacts in a pipeline; **EC2 Image Builder** builds AMIs and container images (Week 2).

Code signing:

- **AWS Signer + Lambda code signing configuration:** Lambda only accepts deployment packages **signed by trusted signing profiles**. The policy can **Enforce** (reject unsigned / invalid) or **Warn**.
- Container images: **AWS Signer + Notation** (see ECR).

8.4 Third-party CI tools

GitHub Actions to AWS without access keys (OIDC):

1. Create an **IAM OIDC identity provider** for `https://token.actions.githubusercontent.com` (audience `sts.amazonaws.com`).
2. Create an IAM **role** whose trust policy allows `sts:AssumeRoleWithWebIdentity` from that provider, **restricted** to your repository and branch.
3. In the workflow, use the `aws-actions/configure-aws-credentials` action with the role ARN (`permissions: id-token: write`). The job gets **temporary credentials**; **no access keys are stored in GitHub**.

```
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::111122223333:oidc-provider/token.actions.githubusercontent.com"
    },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:my-org/payments:ref:refs/heads/main"
    }
  }
}
```

- **Always restrict** `sub` (repository and branch / environment). Without it, **any** GitHub repository could assume the role.
- The same OIDC pattern works for **GitLab CI**, **Bitbucket Pipelines** and others.
- Need VPC access or bigger machines for GitHub Actions jobs: **CodeBuild-hosted GitHub Actions runners** (see CodeConnections).

Jenkins:

- **CodeBuild plugin for Jenkins:** Jenkins sends build jobs to **CodeBuild**, so builds scale without managing a big Jenkins agent fleet.
- **CodePipeline Jenkins action:** a pipeline Build or Test action runs a Jenkins job (the Jenkins plugin polls CodePipeline for jobs).
- Jenkins on **EC2 or ECS** uses an **instance profile / task role** - never stored access keys. Agents can run on Spot (EC2 Fleet plugin).
- **Jenkins on-premises** needing AWS access without long-term keys: **IAM Roles Anywhere** (X.509 certificates from your CA, exchanged for temporary credentials).

Custom actions in CodePipeline:

- For tools with no built-in action, a **custom action** is registered and a **job worker** you run calls `PollForJobs`, does the work, then `PutJobSuccessResult` / `PutJobFailureResult`. (Most use cases are now simpler

with Lambda, Step Functions or the Commands action.)

8.5 Exam traps: quick table

Scenario	Answer
Manual approval before production	Continuous delivery (not continuous deployment)
Every passing change goes to prod with no human step	Continuous deployment
Deploy unfinished features safely, turn on later	Feature flags (AWS AppConfig) - trunk-based development
Developer mistake affected production resources	Separate accounts per environment + central pipeline account
Prod build differs from what was tested	Build once, deploy many - promote the same artifact / image digest
Tests fail but the build passes	Command hides the exit code (an 'or true' fallback, or a pipe) - fix exit codes, `set -o pipefail`
Smoke test with curl passes even on HTTP 500	`curl --fail` (non-zero exit on HTTP errors)
CodeDeploy validation script must fail the deploy	Non-zero exit code in the ValidateService hook
Load test with thousands of users from several Regions	Distributed Load Testing on AWS (staging stage)
Test on real phones and browsers	AWS Device Farm test action
Continuously test user journeys in prod, roll back on failure	CloudWatch Synthetics canary + alarm > CodeDeploy / pipeline rollback
Complex test workflow with parallel steps and retries	Step Functions invoke action
Chaos / resilience test before prod	AWS FIS experiment in the staging stage
Release artifacts must be undeletable for 7 years	S3 Object Lock compliance mode (+ versioning)
Old artifact versions cost too much	S3 lifecycle: expire noncurrent versions
Only signed Lambda code may be deployed	AWS Signer + Lambda code signing configuration (Enforce)
GitHub Actions deploys to AWS without stored keys	IAM OIDC provider + role (AssumeRoleWithWebIdentity)
OIDC role must only work for main of one repository	Trust condition on token.actions.githubusercontent.com:sub
Jenkins can't keep up with build volume	CodeBuild plugin for Jenkins
On-prem Jenkins needs AWS access without keys	IAM Roles Anywhere
Tool with no built-in pipeline action	Custom action + job worker (or Lambda / Step Functions / Commands)

8.6 Hands-on lab (about 1.5 hours)

1. **Exit codes:** in a CodeBuild project, run `pytest || true` with a failing test - see the build pass. Remove `|| true` - see it fail. Try `curl https://httpbin.org/status/500` vs `curl --fail ...`.
2. **Synthetics:** create a CloudWatch Synthetics canary (heartbeat blueprint) against a test URL, and an alarm on its success percentage.
3. **Object Lock:** create a bucket with Object Lock, upload an artifact with a 1-day governance retention, try to delete it.
4. **GitHub Actions OIDC:** create the OIDC provider and a role restricted to your repo's main branch; run a workflow that calls `aws sts get-caller-identity`.
5. **Lambda code signing:** create a Signer profile and a code signing configuration (Enforce); try to deploy an unsigned ZIP.
6. Clean up: canary, alarm, bucket (after retention), role, provider, Signer profile.

8.7 Check yourself

1. What is the difference between continuous delivery and continuous deployment?
2. Tests fail in CodeBuild, but the build is green. Name two likely causes.
3. Why should production never get a rebuilt artifact?

4. How does a GitHub Actions workflow get AWS credentials without access keys, and what must the trust policy restrict?
5. Release artifacts must be impossible to delete for 7 years, even by administrators. What do you use?

Answers:

1. Continuous delivery has a **manual approval** before production; continuous deployment releases **automatically** when all tests pass.
2. The command hides the exit code - `|| true`, or a **pipe** without `set -o pipefail` (e.g. `| tee`).
3. A rebuild can include **different dependencies**, so production would run code that was never tested. Promote the **same artifact / image digest**.
4. An **IAM OIDC identity provider** for GitHub and a role assumed with **AssumeRoleWithWebIdentity**; restrict `token.actions.githubusercontent.com:sub` (repository and branch) and `aud`.
5. **S3 Object Lock in compliance mode** (with versioning) and a 7-year retention period.